

Exploring Peopleware in Continuous Delivery

Pauli Kärpänoja
Solita, Inc.
Tampere, Finland
pauli.karpanoja@solita.fi

Timo Lehtonen
Solita, Inc.
Tampere, Finland
timo.lehtonen@solita.fi

Antti Virtanen
Solita, Inc.
Tampere, Finland
antti.virtanen@solita.fi

Tommi Mikkonen
Tampere U. of Technology
Tampere, Finland
tommi.mikkonen@tut.fi

ABSTRACT

Traditionally, releasing new software has been a fragile and painful procedure. This view has been challenged by a new approach to software deployment, where the goal is to always be able to deploy the system. The transition from manual releases to instant deployments requires a high degree of automation. Furthermore, the transition requires a new mindset, where both developers and operators act together to deliver value to end users. As this process involves humans and cooperation, developers and their attitude towards the new way of working is important. To this end, in this paper we study the developer perspective of applying continuous delivery in the light of interviewing practitioners working for projects where continuous delivery practices are applied. More precisely, we place the focus on how continuous delivery practices affect software developers and what are the prerequisites for achieving continuous delivery. Based on our research, higher quality and other benefits of continuous delivery can be achieved by giving developers more responsibility. Still, the added responsibilities do not necessarily increase stress, but can actually decrease it, and at the same time improve motivation and job satisfaction.

Categories and Subject Descriptors

H.1.2 [User/Machine Systems]: Human factors; D.2.6 [Programming Environments]: Programmer workbench

Keywords

Lean development, peopleware, lead time, deployment pipeline, continuous delivery.

1. INTRODUCTION

Traditionally, releasing new software has been a fragile and painful procedure. Due to the required investment in work and resources, deployments to production have been

rare, and a lot of time has been spent solving seemingly unpredictable problems. This situation is challenged by a new approach to software deployment called continuous delivery [1], where the goal is always to be ready to release and deploy the system. Obviously, this requires an automated tool chain for packaging, testing, and deploying the software, to ensure the latest working version of the software can be deployed at any given moment.

While the above mechanistic view to software development can be characterized as technically feasible, an important aspect has been largely overlooked – the people operating the system and producing the new software, or peopleware [2, 3]. The transition from large, development-heavy releases to small changes made to the system frequently by individual developers requires both a new mindset as well as new techniques and tools that the developers must master. Furthermore, maintaining the deployment pipeline, or, in more general terms, a tool chain that enables continuous delivery, must also be accounted for. These issues are driving the DevOps phenomenon [4] which is not merely a technical solution or a set of tools, but rather a new way of organizing responsibilities and work.

In this paper, we study the developer perspective of applying continuous delivery in a software project. The exact research questions we seek to answer are as follows:

- RQ1: How continuous delivery practices affect software developers?
- RQ2: What are prerequisites for continuous delivery?

The study is based on a series of interviews of practicing software developers who have been previously involved in a more traditional setting, but now work in projects where continuous delivery is applied. The interviews were all executed at Solita Inc., a Finnish software company that specializes in web software and business intelligence.

The rest of this paper, based on results gathered in [5], is structured as follows. In Section 2, we discuss the background of this paper. In Section 3, we address the research approach used to collect empirical data for the paper. In Section 4, we present our results based on the data. In Section 5 we present an extended discussion regarding our observations. In Section 6 we draw some conclusions.

2. BACKGROUND AND RELATED WORK

The goal of continuous delivery is to enable continuous flow of value adding software artifacts from the develop-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

XP '16 Workshops, May 24 2016, Edinburgh, Scotland UK

© 2016 ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2962695.2962708>

ment to the actual production use with a quality assurance. Fowler [1] presents continuous delivery as a set of tools and practices to implement software in such a way that it can be released to production at any time. To this end, four criteria for continuous delivery are presented. Firstly, software is deployable throughout its lifecycle. Secondly, the development team places a higher priority on keeping the software deployable than on working with new features. Thirdly, an automated, rapid cycle produces feedback regarding production readiness after any changes to the software. Finally, any developer can perform a push-button deployment of any version of the software to any environment. To meet these criteria, automation plays a big role – configuring all environments and the deploying software should all be automated and traceable, called infrastructure-as-code (IaC) [6].

Continuous deployment takes the process one step further from continuous delivery, and makes every change go through the pipeline and automatically delivers the results into the production environment [1]. Thus, continuous delivery is a prerequisite for continuous deployment. A closely related concept, the DevOps phenomenon [4], can be described as a set of practices whose goal is to shorten the commit feedback cycle without compromising quality [7]. In other words, it is not merely a technical solution or a set of tools, but a new way of organizing responsibilities and work. However, the term is also used as a catch-all term that can cover several meanings, including at least the following: having development teams and operations speak to each other; allowing automatic deployments; having development teams to be responsible for production error situations [7].

Regardless of the distinction between delivery and deployment, performing continuous software development requires a pipeline that uses an automated set of tools from code to delivery. These tools ensure that each stakeholder gets a timely access to what they need. It is important to notice that the automated pipeline is not about software going into production without any operator supervision, but rather the pipeline provides a feedback loop to each of the stakeholders from all stages of the delivery process. Moreover, as the software progresses through the pipeline, different stages can be triggered for example by operations and test teams by the click of a button.

In addition to automation, a close, collaborative working relationship between everyone involved in delivery is needed. Therefore, software developers' motivation is an important factor in continuous deployment and delivery. The majority of studies on motivation in software engineering reports that software engineers are distinct from other occupational groups with respect to motivation [8]. This underlines the importance of taking the developers into account in any software related activity. To this end, the concept of Developer Experience (DX) aims at providing an intuitive abstraction of the huge variety and quantity of human factors that influence software developers and the outcomes of software engineering [9]. While a related concept, User Experience (UX), considers the context of users, DX considers the context of development and developers. In general, DX includes aspects beyond software development tools, such as development processes, modelling methods, and other means of structuring software engineering tasks.

In general, the work in DX culminates in results presented in [3]; software engineering involves a number of issues related to humans, cooperation, and teamwork. Therefore,

understanding the changes needed in peopleware when migrating from a more traditional model of releasing software to continuous delivery and deployment must be considered. In their paper [10], Olsson et al. describe a model and a roadmap for an R&D organization to achieve continuous delivery from the organization's point of view. The paper covers peopleware briefly, noting for example that it is important to "have mechanisms for team formation and for teams to empower and self-direct themselves".

A case study of a gaming company [11] goes deeper and explains why they had little difficulty selling the continuous delivery approach to their engineers. They describe their engineers as eager to try out new things and constantly reviewing the practices of the most advanced companies, such as Etsy. Their engineers saw the benefits of continuous delivery clearly already and had no resistance to change in general. The Solita engineers involved in projects covered by this paper are in many ways similar in this respect to the employees mentioned in [11]. The main difference between Solita's situation and theirs is that Solita does not have the constant stress of "scary Saturdays", mainly because the nature of the business as well as the customer base are different. A gaming company has more hard deadlines, and the cost of delaying a deployment is different. This gives Solita's engineers a bit less incentive to adopt new practices.

3. RESEARCH APPROACH

The research was carried out by interviewing three people from three different development projects, each of which had been in production at least six months prior to the interviews and exercising the continuous delivery practices. Unlike at Rally Software [11], these projects did not move into continuous delivery after starting with a more traditional approach, but in contrast adopted the practices early on. The interviewees were lead designers, or software developers responsible for the technical architecture as well as feasibility and quality of the solution. The interviewees were all experienced and highly skilled persons, who had years of experience in various ways of doing software development as well as enough experience in continuous delivery practices to reasonably compare, reflect, and analyze the differences.

Each interview was conducted as a semi-structured interview one-on-one. The interviewer had no formal authority over the interviewee, and the discussion was confidential to ensure the interviewees could speak freely. The approximate length of one interview was two hours. All interviews were recorded and the results were gathered up afterwards from the recordings.

The characteristics of the involved projects, all developing web applications, are described in Table 1. All the project teams – in this case, one per project – collaborate closely with their customer organizations, and the products were designed and implemented for some specified occasion according to customer needs. All projects are small, involving 4 to 7 persons working as a single team. The product of each project, despite the dependencies to databases and other technical artefacts, was one single application which was deployable as a standalone system. In each system, the number of build artifacts is one, i.e., the deployment consists of a single installable package. Deployments per month varies from average of two to seven. Case project A and B deployed when the team felt that they should deploy, so the deployment frequency varied a lot. In case project C,

customer wanted the developer to perform the deployment from the customers office. The rest of the items in Table 1 are self-explanatory.

	Case A	Case B	Case C
Software type	Commercial	Commercial	Internal
Team members approximately	7	4	4
Programming language	Clojure, JavaScript	Scala, JavaScript	Java, JavaScript
Lines of Code	60 000	14 000	33 000
Automated end-to-end tests with actual browser	Yes	No	Yes
Automated configuration management (IaC)	Yes	Yes	No
Daily possibility for production deployment	Yes	Yes	Yes
One-step deployment to production (incl. DB-migrations)	Yes	Yes	Yes
Deployments per month (average)	7	5	2

Table 1: Characteristics of the studied projects.

All the interviews were conducted in Finnish, the mother tongue of all persons who participated. The translation to English was done for this research paper by its authors. The interviews also involved topics that were not directly related to peopleware aspects of software development. These fall beyond the scope of this paper, and are not analyzed or discussed here.

The goal of the interviews was to examine the results of adapting continuous delivery practices. The interviews focused on how the practices had been put to use and what were the consequences in respect of for example product quality, effectiveness and readiness to deliver new value. Therefore the topics that were discussed were not always directly associated with peopleware aspect. All the results are reported in the Masters’ Thesis [5]; in this research paper, we place the focus on the results related to peopleware only.

Each interview addressed the following topics:

- The size and type of the project and personnel involved in the project.
- How the version control system was used.
- How deployment pipeline was implemented and used on a daily basis.
- The role of continuous integration.
- Configuration management.
- Test automation and practices.
- Production deployment.
- Readiness to act when new requirements arise.
- Feedback (technical, customer, end-user).
- Motivation, stress, quality and unity.

4. RESULTS

All three projects covered in this study had fully automated their deployment pipelines and processes. Figure 1 presents a simplified deployment pipeline that illustrates how changes in the code are delivered to all environments

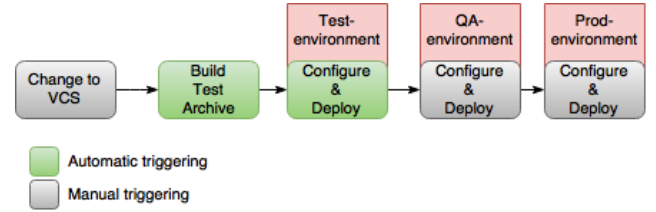


Figure 1: Simplified deployment pipeline.

through automated phases. While there were some differences on exactly how each project fulfilled the phases and practices introduced in [1], the main idea was identical: Every change to the code base triggers a automated process, which builds the application and executes tests against the code or the built artifact. If this phase succeeds, the build is automatically deployed to test environment. Any version deployed to the test environment is a release candidate and may be promoted to QA environment, and after that, into production. Deployments to all environments were scripted and automated, but QA and production deployments were triggered manually. If any changes should be made to the server configurations during the deployment, the same changes should also be automatically accounted for during the application deployment.

4.1 It’s All about Automation

Although every project had not adapted all continuous delivery practices – one project lacked in test automation coverage, and another in automated configuration management – they had all achieved a state where production deployment would have been viable even daily if wanted. Regardless of how completely all the practices were adapted, readiness to production deployment was achieved by automating the deployment pipeline and adapting certain policies to the development process to keep the main branch deployable.

Because the software was built, tested, and deployed to test environment automatically after each change, the development team had high trust for the application to work expectedly in production. Development relied on the deployment pipeline and any failures were fixed immediately to get the build back to the stable state. Programmers were used to writing and modifying tests while introducing new features. Significantly, writing tests was not considered painful – as a matter of fact, they even felt that the shelter brought by the tests was so precious they would like to spend even more time writing tests.

Especially automated end-to-end tests that are run with real browsers were mentioned as being a pain for the developers at times. Though important, such tests may take more than an hour to run and may fail occasionally without any logical reason because of the unreliability of the test frameworks being used. Often some extra time was spent to improve tests and achieve better reliability. One interviewee told that they stopped writing new browser tests for other than just core functionality at one point. When the development team is responsible for the quality, they clearly understand the benefits and importance of tests and the developers take tests more seriously.

Merging any code to the master branch means that the author considers it ready for production. When the programmers were responsible for testing their own implementation,

they also had to trust their colleges that their changes are deployable, too. This created a unified team that was ready to take responsibility for the whole product.

Benefits of automating the infrastructure configurations are obvious when it comes to controlling the whole system. However, just like letting developers be in charge of tests and quality, letting them manage the infrastructure gives them more responsibility and control over the system. Complete control of the configurations and environments reduced the risk of problems occurring in production and therefore increased the team's faith in the product's quality. When teams had more responsibility, they clearly did not consider themselves as just programmers but rather as full members in a solid team.

Production deployments were so stable, that one team had automated it to happen at night, though they made the decision during the day. When deliveries are frequent and automated, the deployment becomes a normal process which does not require anyone to be monitoring or even awake at the time. None of the case projects saw need for long maintenance breaks or targeting the deployment for evenings or weekends. Deployments were done during standard working hours and required no overtime or particular scheduling.

4.2 Readiness to Production Deployment

Being able to deploy a new version to production in a matter of hours also influenced how tense the developers were about bugs and other issues. Two interviewees said that bugs rarely occur in production and one stated that despite the decent test coverage they are quite common. In all cases, the developers did not feel any stress due to bugs, because fixing and deploying the fixes to production was so straightforward.

As the master branch should always be deployable and check-ins to the master branch should be done several times a day, incomplete functionality needs special handling in production. In two cases, this was solved with feature flags [1], and new functionality was hidden with environment-specific setting until ready for use. This convention surprisingly led to fewer version reverts when some problems occurred in the new functionality – instead of reverting older version, the flawed functionality was simply disabled in production.

The developers were satisfied and proud of the situation where they could technically deploy to production whenever needed, but the interviewee in project C mentioned confusion about simultaneous multiple versions of the same feature. When the deployments were only made on particular necessity, the deployments were not always that frequent in both projects B and C. Therefore it was difficult to remember how the version in production was different from the current development version. Infrequent deployments in project B also increased worries that the next production deployment would fail. This might also raise the bar for deployments that are not strictly necessary, which would further erode the trust. Therefore to make production deployments normal non-stressful process for the developers, deployments should be automated and, importantly, happen frequently.

4.3 Impact on Commitment

All interviewees stated that their motivation was high. Everyone said that the opportunity to use and learn mod-

ern technologies increases motivation. Typically technical people like new and fancy stuff, so letting them to use the technologies and methods they prefer improves satisfaction.

Larger responsibility was also said to increase motivation. However, one interviewee also pointed out that it increased stress. When the development team is also responsible for server configurations, testing and quality, wider level of expertise from the programmers is required. On the other hand, two interviewees said that when some integration components or servers were not accessible, they sometimes felt careless if problems were related to these components. Giving the development team more access and responsibility seems to improve motivation and commitment to the project and simultaneously to the organization.

5. DISCUSSION

Practices of continuous delivery are mostly proven to have positive impact on the quality of the system, stability of the deployments, feedback frequency and readiness to act if requirement for changes arise [11, 12]. However, adapting the practices also affects the peopleware aspect as related to the development team.

In general, people are known to do a better job and being more productive when motivated and committed to the organization and project they are working on [2]. To this end, the practices of continuous delivery are all about giving more responsibility to the development team – not only for the software and its quality but also for server configurations and the delivery, which sometimes are considered as operational rather than development related tasks.

Since applying the practices of continuous delivery improved quality and stability of the system, the developers felt comfortable working on the project. They did not work overtime or feel stress over deployments or bugs that would appear in the production, because they were always able to deploy fixes to production – immediately, if needed.

Letting developers, operations, and testers work together – or, better yet, making them the same people – creates a unified team that takes full responsibility for the whole product. Although every programmer not necessarily needs to be an expert in all the fields concerned to continuous delivery, the development team must have members that are skilled enough to master the whole delivery process. All this clearly positively affects the liability of the people and encourages them to be in charge for the system and makes them prouder of their work. People who are proud of their work also feel a need to maintain high quality.

Good programmers are always keen to try out new technologies and improve their craft. Thus, letting the developers use modern tools and learn new methods of working makes them satisfied and pleased to work in the project. This should also be the will of every software organization – train their staff during the regular work to keep them professional and qualified.

To summarize, the direct answers to the research questions of this paper are as follows:

- RQ1: How continuous delivery practices affect software developers?
- Practices improve team's motivation and commitment to the project and organization, contributing to building a unified team that is fully responsible and proud of their work.

- Positive impacts on product quality, stability of the deployments and readiness to act to new requirements or bugs reduce stress and the need for overtime work.
- RQ2: What are prerequisites for continuous delivery?
- Skilled people that are qualified enough to take responsibility of the whole system, infrastructure and delivery; a team that is prepared for learning new ways of working; and developers who are not afraid of larger responsibility.
- Commitment to follow agreed procedures are required from everyone involved in the project.

To study the peopleware aspect of continuous delivery we consider interviews a valid research approach as people in this context can not be easily measured with a statistical approach in a meaningful way. The interviews were made by one researcher only, who also participated in one of the projects, and no thorough transliteration was made for the interview. This was compensated by peer reviewing the interview data and results as a part of the supervision of [5] by all the authors of this paper. In addition, the set of projects that was considered is obviously too small for drawing generic conclusions. Still, we feel confident that the findings would be similar if we were to interview other software projects in Solita. However, as, Solita's developers and engineers are eager to try out new things and open to change and improvement, the results should be viewed keeping this in mind. More conservative people under similar circumstances might feel negatively about the change and experience more stress, even if the new way of doing things would bring objective benefits to the whole team and the customer.

6. CONCLUSIONS

The process of being able to continuously deliver new features to end users has several other benefits than just getting faster feedback. The new way of working irrevocably changes how software developers perform their tasks and feel about the project and organization.

In this paper, we have studied the prerequisites of this way of working as well as how the practices affect the projects and developers. The results are based on interviews of developers leading industrial projects, where continuous delivery is exercised.

The practices of continuous delivery makes a more unified team that is ready to take full responsibility that the system works as expected. People are more motivated and committed when given more responsibility and freedom to use modern technologies and working techniques. Moreover, practices that improve quality and readiness to act on problems in production, also appear to lessen stress. Moreover, the longer the time that has passed since the latest production deployment, the less confident the developers feel to make the next production deployment.

Overall, we believe that continuous delivery and continuous deployment will begin to dominate over sporadic non-automated delivery of software. To reap the benefits and succeed in this transition, organizations should understand this improvement may require profound changes in the way the work is organized, depending on the current situation of the organization. Achieving continuous delivery is no longer primarily a technical challenge, rather it challenges the way

people see and organize their work and responsibilities. If the same team is held responsible for both delivery and production – and also given sufficient autonomy and resources – they will find a way to implement the necessary changes and technical solutions. Moreover, based on this study, we feel confident that such change can make the team feel positive and empowered despite the new responsibilities and demands.

Acknowledgements

The authors wish to thank Solita for providing the opportunity to perform the interviews addressed in this paper. In addition, we wish to thank Digile/Tekes Need for Speed program for supporting this research.

7. REFERENCES

- [1] M. Fowler, "Continuous delivery," <http://martinfowler.com/bliki/ContinuousDelivery.html>, retrieved: November 2014.
- [2] C. G. Andrew, "The peopleware paradigm." *Hospital materiel management quarterly*, vol. 18, no. 1, pp. 47–60, 1996.
- [3] T. DeMarco and T. Lister, *Peopleware: productive projects and teams*. Addison-Wesley, 2013.
- [4] P. Debois, "Devops: A software revolution in the making," *Cutter IT Journal*, vol. 24, no. 8, 2011.
- [5] P. Kärpänöja, "Continuous delivery in customer driven software project," Master's thesis, Tampere University of Technology, December 2015.
- [6] W. Hummer, F. Rosenberg, F. Oliveira, and T. Eilam, "Testing idempotence for infrastructure as code," in *Middleware 2013*. Springer, 2013, pp. 368–388.
- [7] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015.
- [8] S. Beecham, N. Baddoo, T. Hall, H. Robinson, and H. Sharp, "Motivation in software engineering: A systematic literature review," *Information and software technology*, vol. 50, no. 9, pp. 860–878, 2008.
- [9] F. Fagerholm and J. Münch, "Developer experience: Concept and definition," in *Proceedings of the International Conference on Software and System Process*, ser. ICSSP '12. Piscataway, NJ, USA: IEEE Press, 2012, pp. 73–77. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2664360.2664372>
- [10] H. H. Olsson, H. Alahyari, and J. Bosch, "Climbing the "stairway to heaven" - a multiple-case study exploring barriers in the transition from agile development towards continuous deployment of software." in *EUROMICRO-SEAA*, V. Cortellessa, H. Muccini, and O. Demirörs, Eds. IEEE Computer Society, 2012, pp. 392–399. [Online]. Available: <http://dblp.uni-trier.de/db/conf/euromicro/euromicro2012.html#OlssonAB12>
- [11] S. Neely and S. Stolt, "Continuous Delivery? Easy! Just Change Everything (Well, Maybe It Is Not That Easy)." in *AGILE*. IEEE Computer Society, 2013, pp. 121–128. [Online]. Available: <http://dblp.uni-trier.de/db/conf/agile/agile2013.html#NeelyS13>
- [12] L. Chen, "Continuous delivery: Huge benefits, but challenges too," *Software, IEEE*, vol. 32, no. 2, pp. 50–54, 2015.